Plink: Verified Constraint Generation for the Plonk zkSNARK

ANONYMOUS AUTHOR(S)

A zkSNARK is a cryptographic primitive that allows a *prover* to convince a *verifier* that they know some private piece of information satisfying a given mathematical relation $\mathcal{R}$; however, zkSNARKs require that this relation $\mathcal{R}$ is encoded as an algebraic *constraint system*, where each constraint is a polynomial equation. Such an encoding is as low-level and unintuitive as machine code, making writing the equations tedious and error-prone. For this reason, high-level Domain-Specific Languages (DSLs) have been developed to specify the relations and translate them into algebraic constraints. However, even with these high-level DSLs, users can still write incorrect programs or correct programs can be compiled to an incorrect constraint system; in either case, the resulting constraint system may be exploited by malicious users.

In this paper we present a *Programming Language for INtegrity and Knowledge* (Plink), a novel formally verified and strongly-typed DSL for encoding relations as constraint systems for Plonk, a widely used zkSNARK. Plink relies on *refinement types* to both formally specify and statically verify high-level DSL programs and certify the equivalence between the user-written program and the produced constraint system. We use Plink to implement the Poseidon2 and SHA-256 hash functions and formally verify some of their components, showing that it is expressive enough to encode commonly used cryptographic functions.

1 Introduction

A Zero-Knowledge Proof (ZKP) system is a cryptographic primitive that allows one party (the *prover*) to convince another (the *verifier*) that they know some private piece of information satisfying a given property—in the form of a mathematical relation $\mathcal{R}$—without revealing any other information aside from this fact. A powerful class of ZKPs are *Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge (zkSNARKs)*. One of their key features is their general-purpose expressiveness, which enables proving any NP relation $\mathcal{R}$. This versatility has led to the widespread adoption of zkSNARKs across various domains, such as scalability in blockchains [barryWhiteHat 2018; Bonneau et al. 2020], privacy-preserving cryptocurrencies and smart contracts [Aleo Network 2025; Ben Sasson et al. 2014], and machine learning [Balbás et al. 2023; Lagrange Labs 2025; Lee et al. 2024].

The cryptographic protocols implementing zkSNARKs require that the relation $\mathcal{R}$ to be proven is encoded as an *arithmetic circuit*, which is generally described by an algebraic *constraint system* in a large finite field, where each constraint is a polynomial equation. Such an encoding is very low-level and very different from the mental model that developers have, roughly analogous to machine language, which makes writing the equations unintuitive, tedious, and error-prone. This state of affairs has sparked interest—both in academia and in the industry—in developing high-level Domain-Specific Languages (DSLs) to specify the relations and *compilers* to translate them into algebraic constraints [Bellés-Muñoz et al. 2022], in a process known as *arithmetization*.

Even when using these high-level DSLs, it is easy for practitioners without a strong background in cryptography to end up with incorrect constraint systems. This is problematic because these incorrect constraints can be exploited by a malicious user to bypass the security guarantee of the zkSNARK, constructing convincing proofs of false statements. In recent work, Chaliasos et al. [2024] carried out the first systematic study of known vulnerabilities in ZKP systems, providing a taxonomy for classifying them and pointing out some common pitfalls. Of the 141 vulnerabilities they analyzed, 99 were related to errors in the **description** (or programming) of the arithmetic circuit on the part of the developer and 6 were related to bugs in the **compilation** of the description (as a DSL program) into the actual constraints, which are both part of the arithmetization process.

To help mitigate this kind of vulnerabilities in ZKP systems, there is an increasing interest in applying formal methods to the underlying problem of encoding mathematical relations as systems of polynomial equations [Albert et al. 2022; Liu et al. 2024; Provable Inc 2025; Stronati et al. 2024].

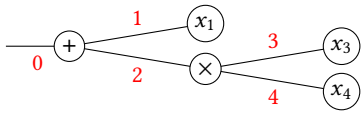
In this work, we focus on the Plonk [Gabizon et al. 2019] zkSNARK, which is a popular protocol especially due to its efficient prover performance and constant-size proofs. Notably, Plonk is currently adopted in many systems—including Zcash [Ben Sasson et al. 2014], Mina [Bonneau et al. 2020], Polygon [Polygon Labs 2023], zkSync [Matter Labs 2023], the Dusk Network [Maharramov et al. 2021], Anoma [Goes et al. 2022], and Midnight [Midnight Network 2024]—where it plays a crucial role in securing significant financial assets, making the need for high-assurance guarantees on the correctness of its constraint systems particularly urgent.

Concretely, we introduce a novel DSL called Plink (*Programming Language for INtegrity and Knowledge*) that assists in the correct arithmetization process for the Plonk zkSNARK. Plink is a formally verified, strongly typed, and declarative programming language based on *refinement types* [Jhala and Vazou 2021]. As such, it helps to mitigate common vulnerabilities in ZK proof systems in two ways: firstly, as a typed and high-level language, it makes it more difficult to describe an arithmetic circuit incorrectly. Further, it offers the necessary infrastructure for the users of the language to formally verify programs written in it through the use of refinement types, by adding precise specifications to the code in the form of logical predicates to be checked automatically at compile time. By relying on such an expressive type system one can, for example, define that a Plink value has a vector type, specify conditions on the lengths of vectors, or even specify that a function on vectors preserves their length. This way, the compiler can detect errors in the description of the circuit automatically. Secondly, it provides strong, formal guarantees that the compiler correctly encodes the user-written program as polynomial equations: we mechanically prove using Liquid Haskell [Vazou et al. 2014]—a program verifier for Haskell based on refinement types—that the Plink compilation process preserves the satisfiability of the constraints, minimizing the chance of errors in the compiler.

To summarize, this work makes the following key contributions:

- We introduce Plink, a strongly typed, declarative DSL embedded in Haskell for describing arithmetic circuits to be compiled down to Plonk constraints, assisting developers in the arithmetization process for Plonk (§ 3).
- We define and mechanize, using Liquid Haskell, the security properties of the Plink compiler; notably, semantics is preserved by both the translation (§§ 4.1, 4.2) and the optimization passes (§ 5.2.1), guaranteeing that the soundness of the Plonk protocol is preserved. Importantly, these correctness proofs—around 880 lines of Liquid Haskell code taking 18 minutes to verify—refer to the actual implementation of the compiler, rather than a model of it in an external proof assistant.
- We show how Liquid Haskell's refinement type system allows the user to *formally specify* a wide range of desirable properties of the implemented program that can then be *statically verified* (§ 4.3). For instance, one can formally verify that the length of a vector is a multiple of 512 (so it can be split in chunks of 512 elements) or prove full functional correctness of the `rotate` function on vectors. Crucially, such properties get propagated to the generated constraints (§ 4.3).
- We evaluate the expressiveness of Plink by implementing two cryptographic hash functions and formally verifying some of their components (§ 6). We have chosen Poseidon2 [Grassi et al. 2023]—a recently proposed family of hash functions that is designed to be *arithmetization-friendly*—and SHA-256—a standard and widely used hash function but which is challenging to arithmetize. The implementation consists of 1680 lines of Haskell code with Liquid Haskell annotations: 1140 for Poseidon2, 250 for SHA-256 and 290 for the library functions, which verify in 100 seconds.

Next, in § 2, we give background on Plonk and refinement types and an overview of Plink. In § 5, we discuss the implementation details that are not included in the presentation of the theory. Finally, in § 7, we compare the Plink language with other DSLs that apply formal methods to the problem of encoding mathematical relations as systems of polynomial equations.



$$\begin{cases} 1 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_0 + 0 \cdot (x_1 \cdot x_2) + 0 = 0 \\ 0 \cdot x_3 + 0 \cdot x_4 - 1 \cdot x_2 + 1 \cdot (x_3 \cdot x_4) + 0 = 0 \end{cases}$$

Fig. 1. Arithmetic circuit and constraint system encodings of the relation "$x_0 = x_1 + x_3 \cdot x_4$"

2 PLONK background & PLINK Overview

We begin this section with a brief outline of the proving process in the PLONK zkSNARK (§ 2.1). Then, we showcase the need of a DSL to describe arithmetic circuits through an example and give an informal intuition of the security properties we should expect from a compiler translating high-level relations down to algebraic equations in § 2.2. We show how to write this example in the PLINK language in § 2.3 and we conclude with an overview of the design of the PLINK compiler (§ 2.4).

2.1 Proving with the PLONK zkSNARK

ZKPs are a powerful cryptographic primitive that enables a *prover* to convince a *verifier* that they know some *secret witness* x such that $\mathcal{R}(x; y)$ holds, where y is a publicly known value and $\mathcal{R}$ is a previously agreed-upon relation. The two main security properties of ZKPs are *soundness* (proofs for invalid statements are accepted only with negligible probability), and *zero-knowledge* (the verifier learns no information about the prover's secret beyond the fact that the statement is true). zkSNARKs are an important class of ZKPs that are *general-purpose* (they can prove any relation $\mathcal{R}$ computable in polynomial time).The protocols implementing zkSNARKs require that the relation $\mathcal{R}$ is encoded as an *algebraic constraint system* C; in the case of PLONK, this is a system of equations as in (1), where the coefficients and variables range over some (large) finite field $\mathbb{F}_p$.

$$q_L \cdot x_a + q_R \cdot x_b + q_O \cdot x_c + q_M \cdot (x_a \cdot x_b) + q_C = 0, \tag{1}$$

A system of constraints like these is often best understood as describing an *arithmetic circuit*: each *gate* in the circuit is described by a constraint while the *wires* connecting the gates are represented by the variables. By choosing appropriate values for the coefficients, equations like (1) can encode a wide variety of operations; for instance, setting $(q_L, q_R, q_O, q_M, q_C) = (1, 1, -1, 0, 0)$ encodes addition and $(q_L, q_R, q_O, q_M, q_C) = (0, 0, -1, 1, 0)$, multiplication. Picturing constraint systems as arithmetic circuits makes reasoning about them much easier (see fig. 1). Under this mental model, each gate has two *input wires* and one *output wire*, even if, strictly speaking, the underlying constraints do not have "inputs" or "outputs".

Formally, proof systems like zkSNARKS are implemented by two algorithms P and V, where

- P takes as arguments a pair (x, y) and produces a proof π, and
- V takes the public value y and the proof π and outputs either `accept` or `reject`.

and where both P and V have access to the constraint system C encoding the relation $\mathcal{R}$. The security of the cryptographic protocol relies on the assumption that the verifier V *trusts the constraint system* C (see fig. 4). However, C is too low-level. If the verifier instead wishes to only trust the relation $\mathcal{R}$, it is fundamental that C is *equivalent* to $\mathcal{R}$, in the sense that knowing a pair $(x, y) \in \mathcal{R}$ amounts to knowing a solution to the constraint system and vice versa.

Solutions to the constraint system are relevant because zkSNARKs are designed in such a way that a valid proof π (one accepted by V) can be constructed with knowledge of a solution (which includes the private value x), but it is computationally infeasible to construct one if such a solution does not exist or is not known by the prover.

These solutions take the form of a mapping from variables to their values. The prover algorithm P needs the values of all variables, but in practice many of these can be computed from just a few;

for convenience, it is common that DSLs for describing arithmetic circuits define a *witness generator* to help in computing these values when possible [Bellés-Muñoz et al. 2022; Liu et al. 2024].

2.2 Motivating example

We claim that the following constraint system is one possible encoding of the "addition modulo 32" relation $\mathcal{R}(x_4; x_1, x_2) \triangleq x_4 \equiv x_1 + x_2 \pmod{32}$ using Plonk's constraints, assuming $0 \leq x_1, x_2 < 32$ (in fact, it is the encoding produced by the Plink compiler C described in § 3.4):

$$
\begin{array}{rlrl}
-1 \cdot x_0 + 0 \cdot x_0 + 0 \cdot x_0 + 1 \cdot x_0 \cdot x_0 + 0 = 0 & (a) & 1 \cdot x_1 + 1 \cdot x_2 - 1 \cdot x_3 + 0 \cdot x_1 \cdot x_2 + 0 = 0 & (b) \\
1 \cdot x_4 + 32 \cdot x_0 - 1 \cdot x_3 + 0 \cdot x_4 \cdot x_0 + 0 = 0 & (c) & 1 \cdot x_6 + 2 \cdot x_{13} - 1 \cdot x_4 + 0 \cdot x_6 \cdot x_{13} + 0 = 0 & (d) \\
1 \cdot x_7 + 2 \cdot x_{12} - 1 \cdot x_{13} + 0 \cdot x_7 \cdot x_{12} + 0 = 0 & (e) & 1 \cdot x_8 + 2 \cdot x_{11} - 1 \cdot x_{12} + 0 \cdot x_8 \cdot x_{11} + 0 = 0 & (f) \\
1 \cdot x_9 + 2 \cdot x_{10} - 1 \cdot x_{11} + 0 \cdot x_9 \cdot x_{10} + 0 = 0 & (g) & -1 \cdot x_6 + 0 \cdot x_6 + 0 \cdot x_0 + 1 \cdot x_6 \cdot x_6 + 0 = 0 & (h) \\
-1 \cdot x_7 + 0 \cdot x_7 + 0 \cdot x_0 + 1 \cdot x_7 \cdot x_7 + 0 = 0 & (i) & -1 \cdot x_8 + 0 \cdot x_8 + 0 \cdot x_0 + 1 \cdot x_8 \cdot x_8 + 0 = 0 & (j) \\
-1 \cdot x_9 + 0 \cdot x_9 + 0 \cdot x_0 + 1 \cdot x_9 \cdot x_9 + 0 = 0 & (k) & -1 \cdot x_{10} + 0 \cdot x_{10} + 0 \cdot x_0 + 1 \cdot x_{10} \cdot x_{10} + 0 = 0 & (l)
\end{array}
$$

Each of the equations encodes either an intermediate step in the computation, or an additional constraint to ensure that the rest of the equations do not have "too many" solutions, that is, solutions that could make the computation of some intermediate step incorrect.

Concretely, equation (b) forces the intermediate variable x_3 to have the value $x_1 + x_2$, while equation (c) says that $x_3 = x_4 + 32 \cdot x_0$, where x_0 (whether there is an overflow in the sum) is constrained by equation (a) to have a *boolean* value, i.e. either 0 or 1. Meanwhile, equations (d)–(g) force x_4 to be the binary expansion of a vector $x_{10}x_9x_8x_7x_6$, i.e. $2^4x_{10} + 2^3x_9 + 2^2x_8 + 2^1x_7 + 2^0x_6$, and equations (h)–(l) make sure that all of $x_6, x_7, x_8, x_9, x_{10}$ are boolean. Together, these equations mean that x_4 can be written using 5 *bits*, and therefore $0 \leq x_4 < 32$.

Thus, the above constraint system is encoding $x_4 \equiv x_1 + x_2 \pmod{32}$ as the following conditions:

$$x_1 + x_2 = x_4 + 32 \cdot x_0 \qquad x_0 \in \{0, 1\} \qquad 0 \leq x_4 < 32. \tag{2}$$

The main challenge in formally defining what it means for a constraint system like (a)–(l) to *encode* a relation like $x_4 \equiv x_1 + x_2 \pmod{32}$ is that, in the mathematical relation, only the input and output values (x_1, x_2, x_4) are relevant, while the constraint system necessarily involves all of the intermediate values. Furthermore, it is sometimes the case that some intermediate variables may take several possible values while satisfying the overall mathematical relation.

In this setting, we say that a constraint system C correctly encodes a relation $\mathcal{R}$ when

(1) for all values in $\mathcal{R}$, there are always values for the intermediate variables that satisfy C, and
(2) any solution to C can be restricted to the exposed variables to obtain values that are in $\mathcal{R}$.

2.3 Plink

In the Plink language, the conditions shown in (2) could be written as shown in fig. 2. Note that it is implemented as a Haskell function that takes the "input" values x_1 and x_2 and returns their modular addition x_4. In this Plink program, the most important lines are 8–10, which are a direct translation of the constraints from (2); however, since the result x_4 and the additional variable x_0 are not given as part of the input, they must first be declared as in lines 4–5.

The first two constraints are straightforward to encode with Plink operators. The third, on the other hand, is a bit more involved and it is common enough that it is worth defining another function for it: `withNBits` is a library function that adds the necessary constraints to ensure that its second argument `x4` (representing the modular sum) can be written as a 5-bit number, and it does this by introducing a vector of binary values and asserting that `x4` is equal to its binary expansion.

The lines 6–7 define *hints* for the witness generator; since the variables `x0` and `x4` are declared locally, the only way to compute their value is by defining it in relation to the value of the input variables `x1` and `x2`. With these hints, the compiler can automatically compute the value of all the

```
1  {-@ addMod32 :: x1:FieldDSL p -> x2:FieldDSL p -> PlinkST p (FieldDSL p) @-}
2  addMod32 :: Field p => DSL p -> DSL p -> PlinkST p (DSL p)
3  addMod32 x1 x2 = do
4    let x0 = VAR "overflow" TF
5    let x4 = VAR "sum" TF
6    witnessGenHint x0 (\x y -> if x + y < 32 then 0 else 1) x1 x2
7    witnessGenHint x4 (\x y -> (x + y) `mod` 32) x1 x2
8    assert $ (x1 `ADD` x2) `EQA` (x4 `ADD` (x0 `MUL` CONST 32))
9    assert $ BOOLEAN x0
10   withNBits 5 x4 -- x4 can be encoded using 5 bits, so 0 <= x4 < 32
11   return x4
```

Fig. 2. PLINK program that returns $x + y \pmod{32}$

intermediate values (*witnesses*) that are mentioned in the constraints (a)–(l) just by knowing the values of the input variables, which would be provided by the prover.

Finally, line 11 "returns" the variable that represents the modular addition so it is available elsewhere (either for its value or to be composed within another PLINK expression). Types are defined in lines 1–2, but for now let us focus on line 2, which defines the *Haskell* type of `addMod32`: it takes two PLINK expressions (two Haskell values of type `DSL p`) and returns a third expression that carries some *assertions* with it (to carry the assertions, we implement the `PlinkST p` writer monad). The type parameter `p` represents the underlying finite field of the constraint system. It is constrained with the type class `Field p`, defined as `(Fractional p, Integral p, Ord p)`. These three constraints are sufficient to encode the finite field requirement; in the implementation, we instantiate them with the `PrimeField` datatype from the `finite-field` library [Sakai 2021].

The arguments `x1` and `x2` and the result `x4` represent PLINK expressions of type "field"; line 1 encodes this restriction using refinement types.

2.3.1 Refinement types in PLINK. PLINK has a simple type system, with only support for field elements, booleans and vectors; still, it benefits from the expressiveness of *refinement types* by being implemented as an embedded DSL in Liquid Haskell and relying on its powerful type system.

Liquid Haskell [Vazou et al. 2014] is a formal verifier that checks that the Haskell code satisfies the refinement types specifications written within comments of the form `{-@ ... @-}`. *Refinement types* [Jhala and Vazou 2021] constrain existing types with logical predicates, allowing the programmer to express precise properties not captured by the base type system. In PLINK, refinement types serve to encode the types of PLINK expressions as well as functional correctness properties.

As an example, type annotations in PLINK are written by *refining* the base `DSL` syntactic category, representing expressions, with the type of the expression. For instance, `FieldDSL p` is just an alias for "`DSL p` with a type of *field*"—or, formally, `{v:DSL p | typed v TF}`—and PLINK type checking is reduced to refinement type checking in Liquid Haskell.

A second example of the use of refinement types in PLINK is adding a postcondition to a function by refining the type of its return value. For instance, fig. 3 shows a function that concatenates two PLINK vectors of an arbitrary type `t` together. The *refinement type* (lines 1–2) of this function is stating that, given two programs `u` and `v` that are vectors of `t`, the function `vAppend` will return another vector `w` such that its length is the sum of the lengths of `u` and `v`. When compared to the base type signature `Ty -> DSL p -> DSL p -> DSL p`, the refinement type signature encodes much more information about the function `vAppend`: not only the types of its arguments and result (all vectors of `t`), but also about its behavior. Since the compiler will check that the body of `vAppend`

```
{-@ vAppend :: τ:Ty -> u:VecDSL p τ -> v:VecDSL p τ
            -> {w:VecDSL p τ | vlen w = vlen u + vlen v} @-}
vAppend :: Ty -> DSL p -> DSL p -> DSL p
vAppend τ (NIL _)      ys = ys
vAppend τ (CONS x xs) ys = CONS x (vAppend τ xs ys)
```

Fig. 3. Plink implementation of vAppend

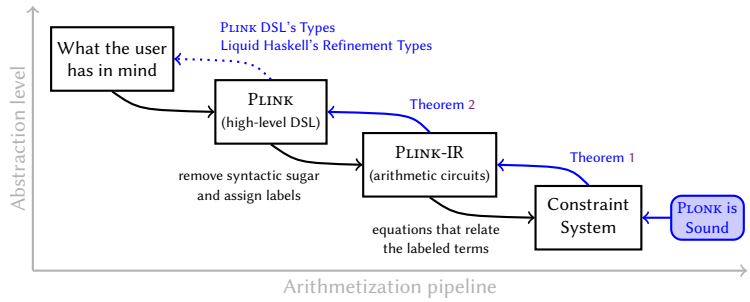


Fig. 4. Plink Arithmetization Pipeline

indeed satisfies the specification, we get a high degree of confidence that it is implemented correctly. Section 4.3 presents some more examples of properties that can be verified with refinement types.

Apart from these two use cases (expressing Plink's types and enforcing functional correctness of the Haskell functions that generate Plink values), we use Liquid Haskell as a theorem prover to mechanically prove the correctness of the compiler.

2.4 Our approach

In the Plink arithmetization pipeline (fig. 4), the first step is describing the arithmetic circuit on the part of the user. This step essentially amounts to coming up with a set of conditions like in (2); in this respect, Plink provides some assistance by letting the user write programs in a typed DSL as well as formal specifications through Liquid Haskell's refinement type system.

The second step of the arithmetization pipeline (the first compiler pass) consists primarily of annotating the high-level Plink program such that every subexpression is given a label, producing a Plink-IR term that describes an arithmetic circuit like the one in fig. 1. Each label corresponds to a variable in the generated constraint system, which is critical in proving that the translation preserves the semantics: the labels provide the means to reference the subexpressions of the program, so that a connection with the variables in the Plonk constraint system can be established. The correctness of this labeling pass is formalized in § 4.2. Additionally, syntactic sugar is removed in this phase and standard compiler optimizations are applied, as described in § 5.2.2.

Finally, the last step is compiling the labeled intermediate representation down to the constraint system in the format expected by the Plonk protocol, as in equations (a)–(l). The correctness of the compilation phase is formalized in § 4.1.

Plink is carefully designed to ensure that Plonk's soundness is propagated to the levels closer to the user. Firstly, the Plonk protocol guarantees that, with overwhelming probability, a prover that is able to construct a valid proof does indeed know a solution to the constraint system. Secondly, theorem 1 states that a set of constraints is satisfiable if and only if its Plink-IR representation is satisfiable. Thirdly, theorem 2 states that a Plink-IR program is satisfiable if and only if the

assertions in the original PLINK program are satisfiable. Combining these three facts, we extend the guarantees of the PLONK protocol to the level of the PLINK DSL: a prover that can construct a valid proof knows a valuation that satisfies the assertions in the high-level program, with overwhelming probability. Even if it is not possible to fully certify that the mathematical relation $\mathcal{R}$ is correctly encoded in the high-level PLINK program, our implementation relies on Liquid Haskell's refinement and PLINK's type system to rule out many common mistakes.

3 The PLINK Language

In this section, we define the technical aspects of the PLINK language: its syntax (§ 3.1), types (§ 3.2), semantics (§ 3.3), compilation procedure (§ 3.4) and witness generation process (§ 3.5).

More concretely, the compilation process is carried out in two steps to facilitate the proof of correctness of the compiler: a first step removes syntactic sugar and labels all subexpressions with integer indices, and a second step compiles the labeled intermediate representation programs down to a constraint system. For this reason, in addition to the PLINK language, we describe here the intermediate representation language PLINK-IR as well as the translations from PLINK to PLINK-IR and from PLINK-IR down to the language of PLONK's constraint systems.

3.1 The PLINK Syntax

The syntax of the PLINK and PLINK-IR languages is presented in figures 5a and 5b, respectively.

Top-level PLINK programs (see fig. 5a) consist of a single expression e and a (possibly empty) list of assertions a^* that add restrictions on the values of program subexpressions. Expressions can be field or boolean constants, variables, arithmetic and boolean operations, equality checks (with a special case for checking equality against the constant 0 for convenience), type casts (from booleans to field values only) and vector literals. Note that variables are always annotated with their declared type.The assertions include non-zero, booleanity, and equality assertions. The types in PLINK are field (F), bool (Bool) and vectors of other types. PLINK values are limited to field and vector constants.

The intermediate representation language PLINK-IR (see fig. 5b) is designed to be close to the high-level PLINK language: expressions can be constants, variables, binary expressions and equality checks (only against the constant 0). There are, however, four important distinctions:

(1) All expressions are labeled with one or two "wire" indices, and so are the assertions that need them. These indices correspond exactly to the PLONK variables that appear in the constraint system after compilation and are used to establish this connection rigorously.
(2) PLINK-IR expressions additionally include *wires*, referred to by their index.
(3) There are no "top-level" programs; instead, PLINK-IR deals with a collection of labeled programs, which can be labeled expressions (useful for their values, either as output values or to be composed further) or labeled assertions (to constrain the possible values of other expressions).
(4) Syntactic sugar (boolean constants and equality checks) and higher-level constructs (type casts, which semantically are a no-op, and vectors) are not included in the intermediate representation.

These features mean that PLINK-IR is a faithful *textual* description of arithmetic circuits with a fixed set of available gates: gates are described by expressions, which relate the wire indices.

As a convention, we always use j to denote the index of the "output wire" of a given labeled expression $\hat{e}$, *i.e.*, the wire holding the value of the expression; we denote by ι the *output wire* function that, when given a labeled expression, returns its output wire index j. Formally,

$$\iota(\mathsf{wire}_j) = j, \quad \iota(c, j) = j, \quad \iota(x^\tau, j) = j, \quad \iota(\hat{e}_1 \odot \hat{e}_2, j) = j, \quad \iota(\hat{e}_1/\hat{e}_2, w, j) = j, \quad \iota(\mathsf{isZero}\ \hat{e}, w, j) = j$$

Note that most labeled expressions in fig. 5b have only one index, but two of them (division and checking equality against 0) have an additional one (denoted as w) that represents a "witness" wire,

Programs	p	::=	program $e\ a^*$
Field Elems.	c	$\in$	$\mathbb{F}_p$
Expressions	e	::=	$c \mid \mathtt{tt} \mid \mathtt{ff} \mid x^\tau \mid e \odot e$
		$\mid$	$e = e \mid \mathsf{isZero}\ e \mid [\,]$
		$\mid$	$\mathsf{boolToF}\ e \mid e :: e$
Operators	$\odot$	::=	$+ \mid - \mid * \mid / \mid \vee \mid \wedge \mid \dot{\vee}$
Assertions	a	::=	$e{\neq}0 \mid e{\in}\mathbb{B} \mid e \doteq e$
Types	τ	::=	$\mathsf{F} \mid \mathsf{Bool} \mid \mathsf{Vec}\ \tau$
Values	v	::=	$c \mid [\,] \mid v :: v$
Valuation	ρ	::=	$\varnothing \mid \rho,\ x^\tau \mapsto c$

(a) Syntax of the Plink language

Labeled Prog.	$\hat{p}$	::=	$\hat{e} \mid \hat{a}$
Wire Indices	j, w	$\in$	$\mathbb{N}$
Labeled Expr.	$\hat{e}$	::=	$\mathsf{wire}_j \mid (c, j)$
(for $\odot \neq /$)		$\mid$	$(\hat{e} \odot \hat{e}, j) \mid (x^\tau, j)$
		$\mid$	$(\hat{e}/\hat{e}, w, j)$
		$\mid$	$(\mathsf{isZero}\ \hat{e}, w, j)$
Labeled Asser.	$\hat{a}$	::=	$(\hat{e}{\neq}0, w)$
		$\mid$	$\hat{e}{\in}\mathbb{B} \mid \hat{e} \doteq \hat{e}$
Labeling Env.	Λ	::=	$\varnothing \mid \Lambda,\ x^\tau \mapsto j$
Valuation	σ	::=	$\varnothing \mid \sigma,\ j \mapsto c$

(b) Syntax of the Plink-IR Language

Fig. 5. Syntax of the Plink and Plink-IR languages.

Typing $\boxed{e : \tau}$

$$\frac{}{c : \mathsf{F}}\ \textsc{T-Fld} \qquad \frac{}{b : \mathsf{Bool}}\ \textsc{T-Bool} \qquad \frac{\tau \in \{\mathsf{F}, \mathsf{Bool}\}}{x^\tau : \tau}\ \textsc{T-Var} \qquad \frac{e_1 : \mathsf{F} \quad e_2 : \mathsf{F}}{e_1 = e_2 : \mathsf{Bool}}\ \textsc{T-Eq}$$

$$\frac{e_1 : \mathsf{F} \quad e_2 : \mathsf{F} \quad \odot \in \{+, -, *, /\}}{e_1 \odot e_2 : \mathsf{F}}\ \textsc{T-BinOpF} \qquad \frac{e_1 : \mathsf{Bool} \quad e_2 : \mathsf{Bool} \quad \odot \in \{\wedge, \vee, \dot{\vee}\}}{e_1 \odot e_2 : \mathsf{Bool}}\ \textsc{T-BinOpBool}$$

$$\frac{e : \mathsf{F}}{\mathsf{isZero}\ e : \mathsf{Bool}}\ \textsc{T-IsZr} \qquad \frac{e : \mathsf{Bool}}{\mathsf{boolToF}\ e : \mathsf{F}}\ \textsc{T-Cast} \qquad \frac{}{[\,] : \mathsf{Vec}\ \tau}\ \textsc{T-Nil} \qquad \frac{e_h : \tau \quad e_t : \mathsf{Vec}\ \tau}{e_h :: e_t : \mathsf{Vec}\ \tau}\ \textsc{T-Cons}$$

$\boxed{\vdash a}$

$$\frac{e : \tau \quad \tau \in \{\mathsf{F}, \mathsf{Bool}\}}{\vdash e{\neq}0}\ \textsc{T-NonZrA} \qquad \frac{e : \tau \quad \tau \in \{\mathsf{F}, \mathsf{Bool}\}}{\vdash e{\in}\mathbb{B}}\ \textsc{T-BoolA} \qquad \frac{e_1 : \mathsf{F} \quad e_2 : \mathsf{F}}{\vdash e_1 \doteq e_2}\ \textsc{L-EqA}$$

Fig. 6. Typing rules for Plink

needed to ensure the correct behavior of the gate. Note also that labeled assertions do not need an "output" index j as they produce no results, and only the nonZero assertion needs a "witness" wire.

3.2 Typing rules

Plink types are relevant for two main reasons: (1) they make it impossible to write certain incorrect programs, and (2) they are used to direct the compilation of variables, since a variable that is declared as *boolean* is only allowed to take the values 0 or 1. The only other way to enforce such a restriction would be for the programmer to add booleanity assertions manually, which is error-prone.

Well-typed expressions are described with the typing judgment $e : \tau$, meaning that expression e has type τ; note that no typing environment is required since variables are type annotated. Similarly, well-typed assertions are denoted with the judgment $\vdash a$, which means that the expressions appearing in assertion a are all well-typed and have types compatible with the assertion. The typing rules, presented in fig. 6, are mostly standard—however, there are two points worth noting:

(1) Variables must be annotated with a type and this type must be *scalar*, that is, either F or Bool, since Plink variables represent wires in the arithmetic circuit and these can only be scalars.
(2) Assertions themselves are *not* typed, but they do require their arguments to be well-typed.

PLINK semantics $\boxed{\rho \vdash e \Downarrow v}$

$$\frac{}{\rho \vdash c \Downarrow c}\ \text{E-FLD} \qquad \frac{}{\rho \vdash \mathtt{tt} \Downarrow 1}\ \text{E-TRUE} \qquad \frac{}{\rho \vdash \mathtt{ff} \Downarrow 0}\ \text{E-FALSE} \qquad \frac{\rho(x^{\mathsf{F}}) = c}{\rho \vdash x^{\mathsf{F}} \Downarrow c}\ \text{E-VARF}$$

$$\frac{\rho(x^{\mathsf{Bool}}) = c \quad c \in \{0,1\}}{\rho \vdash x^{\mathsf{Bool}} \Downarrow c}\ \text{E-VARBOOL} \qquad \frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash e_1 = e_2 \Downarrow \text{if } v_1 = v_2 \text{ then } \mathtt{tt} \text{ else } \mathtt{ff}}\ \text{E-EQ}$$

$$\frac{\rho \vdash e_1 \Downarrow v_1 \quad \rho \vdash e_2 \Downarrow v_2}{\rho \vdash e_1 \odot e_2 \Downarrow v_1 \odot_{\mathbb{F}_p} v_2}\ \text{E-BINOP} \qquad \frac{\rho \vdash e \Downarrow v}{\rho \vdash \text{isZero } e \Downarrow \text{if } v = 0 \text{ then } \mathtt{tt} \text{ else } \mathtt{ff}}\ \text{E-ISZR}$$

$$\frac{\rho \vdash e \Downarrow v}{\rho \vdash \text{boolToF } e \Downarrow v}\ \text{E-CAST} \qquad \frac{}{\rho \vdash [\,] \Downarrow [\,]}\ \text{E-NIL} \qquad \frac{\rho \vdash e_h \Downarrow v_h \quad \rho \vdash e_t \Downarrow v_t}{\rho \vdash e_h :: e_t \Downarrow v_h :: v_t}\ \text{E-CONS}$$

$\boxed{\rho \vdash a}$

$$\frac{\rho \vdash e \Downarrow v \quad v \neq 0}{\rho \vdash e{\neq}0}\ \text{E-NZRA} \qquad \frac{\rho \vdash e \Downarrow v \quad v \in \{0,1\}}{\rho \vdash e{\in}\mathbb{B}}\ \text{E-BOOLA} \qquad \frac{\rho \vdash e_1 \Downarrow v \quad \rho \vdash e_2 \Downarrow v}{\rho \vdash e_1 \doteq e_2}\ \text{E-EQA}$$

Fig. 7. Semantics of the PLINK Language

PLINK-IR semantics (valuation coherence) $\boxed{\sigma \vdash \hat{e}}$

$$\frac{}{\sigma \vdash \text{wire}_j}\ \text{C-WIRE} \qquad \frac{\sigma(j) = c}{\sigma \vdash (c, j)}\ \text{C-FLD} \qquad \frac{}{\sigma \vdash (x^{\mathsf{F}}, j)}\ \text{C-VARF} \qquad \frac{\sigma(j) \in \{0,1\}}{\sigma \vdash (x^{\mathsf{Bool}}, j)}\ \text{C-VARBOOL}$$

$$\frac{\begin{array}{c}\sigma \vdash \hat{e}_1 \quad \sigma(j) = \overline{\sigma}(\hat{e}_1) \odot_{\mathbb{F}_p} \overline{\sigma}(\hat{e}_2) \\ \sigma \vdash \hat{e}_2 \quad \odot \in \{+, -, *, \wedge, \vee, \dot{\vee}\}\end{array}}{\sigma \vdash (\hat{e}_1 \odot \hat{e}_2, j)}\ \text{C-BINOP} \qquad \frac{\begin{array}{c}\sigma \vdash \hat{e}_1 \quad \sigma(w) \cdot \overline{\sigma}(\hat{e}_2) = 1 \\ \sigma \vdash \hat{e}_2 \quad \sigma(j) = \overline{\sigma}(\hat{e}_1) /_{\mathbb{F}_p} \overline{\sigma}(\hat{e}_2)\end{array}}{\sigma \vdash (\hat{e}_1 / \hat{e}_2, w, j)}\ \text{C-DIV}$$

$$\frac{\begin{array}{c}\sigma \vdash \hat{e} \quad \sigma(j) \in \{0,1\} \quad \overline{\sigma}(\hat{e}) \cdot \sigma(j) = 0 \\ \text{if } \overline{\sigma}(\hat{e}) = 0 \text{ then } \sigma(j) = 1 \text{ else } \sigma(j) = 0 \wedge \sigma(w) \cdot \overline{\sigma}(\hat{e}) = 1\end{array}}{\sigma \vdash (\text{isZero } \hat{e}, w, j)}\ \text{C-ISZR}$$

$\boxed{\sigma \vdash \hat{a}}$

$$\frac{\begin{array}{c}\sigma \vdash \hat{e} \\ \overline{\sigma}(\hat{e}) \cdot \sigma(w) = 1\end{array}}{\sigma \vdash (\hat{e}{\neq}0, w)}\ \text{C-NONZRA} \qquad \frac{\begin{array}{c}\sigma \vdash \hat{e} \\ \overline{\sigma}(\hat{e}) \in \{0,1\}\end{array}}{\sigma \vdash \hat{e}{\in}\mathbb{B}}\ \text{C-BOOLA} \qquad \frac{\begin{array}{c}\sigma \vdash \hat{e}_1 \quad \sigma \vdash \hat{e}_2 \\ \overline{\sigma}(\hat{e}_1) = \overline{\sigma}(\hat{e}_2)\end{array}}{\sigma \vdash \hat{e}_1 \doteq \hat{e}_2}\ \text{C-EQA}$$

Fig. 8. Valuation-Coherence of the PLINK-IR Language

We note that there are no typing rules for the PLINK-IR language: since PLINK-IR programs are not written manually, there is limited value in preventing the construction of ill-typed programs. The only way in which types are relevant in the PLINK-IR language is in the variables: type annotations are not erased so that they can be used to add the necessary constraints (cf. fig. 8 and fig. 10).

3.3 Semantics

Semantics for PLINK and PLINK-IR are defined in figures 7 and 8, respectively.

The semantics for PLINK is straightforward: we use the judgment $\rho \vdash e \Downarrow v$ to denote the evaluation of an expression e under a valuation ρ to produce a value v, where valuations ρ are maps from variable *names* to field values as defined in fig. 5a. Simultaneously, it serves to formalize the

Labeling $\boxed{\Lambda \vdash p \rightsquigarrow \hat{p}^* \dashv \Lambda}$

$$\frac{\Lambda \vdash as \rightsquigarrow^* \hat{as} \dashv \Lambda' \quad \Lambda' \vdash e \rightsquigarrow \hat{es} \dashv \Lambda''}{\Lambda \vdash \mathsf{program}\ e\ as \rightsquigarrow \hat{as} + \hat{es} \dashv \Lambda''}\ \text{L-Prog} \qquad \frac{j \text{ fresh}}{\Lambda \vdash c \rightsquigarrow \langle (c, j) \rangle \dashv \Lambda}\ \text{L-Fld}$$

$$\frac{j \text{ fresh}}{\Lambda \vdash \mathsf{tt} \rightsquigarrow \langle (1, j) \rangle \dashv \Lambda}\ \text{L-True} \qquad \frac{j \text{ fresh}}{\Lambda \vdash \mathsf{ff} \rightsquigarrow \langle (0, j) \rangle \dashv \Lambda}\ \text{L-False}$$

$$\frac{\Lambda(x^\tau) = j}{\Lambda \vdash x^\tau \rightsquigarrow \langle \mathsf{wire}_j \rangle \dashv \Lambda}\ \text{L-Var1} \qquad \frac{x^\tau \notin \mathrm{dom}(\Lambda) \quad j \text{ fresh}}{\Lambda \vdash x^\tau \rightsquigarrow \langle (x^\tau, j) \rangle \dashv \Lambda, x^\tau \mapsto j}\ \text{L-Var2}$$

$$\frac{j \text{ fresh} \quad \odot \in \{+, -, *, \wedge, \vee, \dot{\vee}\} \quad \Lambda \vdash e_1 \rightsquigarrow \langle \hat{e}_1 \rangle \dashv \Lambda' \quad \Lambda' \vdash e_2 \rightsquigarrow \langle \hat{e}_2 \rangle \dashv \Lambda''}{\Lambda \vdash e_1 \odot e_2 \rightsquigarrow \langle (\hat{e}_1 \odot \hat{e}_2, j) \rangle \dashv \Lambda''}\ \text{L-BOp} \qquad \frac{\Lambda \vdash e_1 \rightsquigarrow \langle \hat{e}_1 \rangle \dashv \Lambda' \quad \Lambda' \vdash e_2 \rightsquigarrow \langle \hat{e}_2 \rangle \dashv \Lambda'' \quad j, w \text{ fresh}}{\Lambda \vdash e_1 / e_2 \rightsquigarrow \langle (\hat{e}_1 / \hat{e}_2, w, j) \rangle \dashv \Lambda''}\ \text{L-Div}$$

$$\frac{\Lambda \vdash \mathsf{isZero}\ (e_1 - e_2) \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'}{\Lambda \vdash (e_1 = e_2) \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'}\ \text{L-Eq} \qquad \frac{\Lambda \vdash e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda' \quad j, w \text{ fresh}}{\Lambda \vdash \mathsf{isZero}\ e \rightsquigarrow \langle (\mathsf{isZero}\ \hat{e}, w, j) \rangle \dashv \Lambda'}\ \text{L-IsZr}$$

$$\frac{}{\Lambda \vdash [\,] \rightsquigarrow \epsilon \dashv \Lambda}\ \text{L-Nil} \qquad \frac{\Lambda \vdash e_1 \rightsquigarrow \hat{es}_1 \dashv \Lambda' \quad \Lambda' \vdash e_2 \rightsquigarrow \hat{es}_2 \dashv \Lambda''}{\Lambda \vdash e_1 :: e_2 \rightsquigarrow \hat{es}_1 + \hat{es}_2 \dashv \Lambda''}\ \text{L-Cons}$$

$$\frac{\Lambda \vdash e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'}{\Lambda \vdash \mathsf{boolToF}\ e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'}\ \text{L-Cast} \qquad \frac{\Lambda \vdash e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda' \quad w \text{ fresh}}{\Lambda \vdash e{\neq}0 \rightsquigarrow \langle (\hat{e}{\neq}0, w) \rangle \dashv \Lambda'}\ \text{L-NonZrA}$$

$$\frac{\Lambda \vdash e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'}{\Lambda \vdash e{\in}\mathbb{B} \rightsquigarrow \langle \hat{e}{\in}\mathbb{B} \rangle \dashv \Lambda'}\ \text{L-BlA} \qquad \frac{\Lambda \vdash e_1 \rightsquigarrow \langle \hat{e}_1 \rangle \dashv \Lambda' \quad \Lambda' \vdash e_2 \rightsquigarrow \langle \hat{e}_2 \rangle \dashv \Lambda''}{\Lambda \vdash (e_1 \doteq e_2) \rightsquigarrow \langle \hat{e}_1 \doteq \hat{e}_2 \rangle \dashv \Lambda''}\ \text{L-EqA}$$

Fig. 9. Labeling

notion of an assertion a being satisfied under a valuation ρ, described with the judgment $\rho \vdash a$. The satisfiability relation naturally extends from a single element to a list as $\rho \vdash as \iff \forall a \in as, \rho \vdash a$.

Plink-IR makes use of a second kind of valuations, *wire valuations* σ, that map wire *indices* to field values, as defined in fig. 5b. For simplicity, we write $\overline{\sigma} = \sigma \circ \iota$ to denote the function mapping labeled expressions to their output value. For example, given a labeled expression like $\hat{e} \triangleq (\mathsf{wire}_0 + (x^\tau, 1), 2)$ and a valuation $\sigma \triangleq 0 \mapsto 3, 1 \mapsto 4, 2 \mapsto 7$, the output value of $\hat{e}$ is $\overline{\sigma}(\hat{e}) = \sigma(\iota(\hat{e})) = \sigma(2) = 7$.

Figure 8 makes explicit how Plink-IR programs relate the values in the wires of an arithmetic circuit: the semantics of Plink-IR programs is not only defined by their *overall* value in relation to the inputs; instead, we say that a wire valuation σ is *coherent* with a labeled expression $\hat{e}$ (denoted $\sigma \vdash \hat{e}$) if the values it assigns to the wires are in the relation defined by the expression. As before, this semantics induces the notion of an assertion $\hat{a}$ being satisfied by a valuation σ, written as $\sigma \vdash \hat{a}$.

For example, given $\hat{e} \triangleq (\mathsf{wire}_0 + (x^\tau, 1), 2)$, we can see that the valuation $\sigma_1 \triangleq 0 \mapsto 3, 1 \mapsto 4, 2 \mapsto 7$ is coherent with $\hat{e}$ since $\sigma_1(0) + \sigma_1(1) = \sigma_1(2)$, while $\sigma_2 \triangleq 0 \mapsto 3, 1 \mapsto 4, 2 \mapsto 42$ is not.

A particularly interesting rule is C-NonZrA; it states that, in order for a valuation σ to satisfy the assertion $(\hat{e}{\neq}0, w)$, the value of the witness wire w under σ should be the multiplicative inverse of the value of the expression $\overline{\sigma}(\hat{e})$, as this inverse exists if and only if the value is indeed non-zero. Rules C-Div and C-IsZr also include similar premises.

Wire valuations also define the behavior of constraint systems. Given a constraint system C, we say that a wire valuation σ *satisfies* or *is a solution to* C (denoted $\sigma \vdash C$) if each equation in the system is satisfied when substituting variables x_j for $\sigma(j)$. Formally, $\sigma \vdash C$ *iff*, for each equation in C,

$$q_L \cdot \sigma(a) + q_R \cdot \sigma(b) + q_O \cdot \sigma(c) + q_M \cdot (\sigma(a) \cdot \sigma(b)) + q_C = 0.$$

Compiling $\boxed{C : \hat{e} \to CS}$

$$
\begin{aligned}
C(\mathsf{wire}_j) &\triangleq \epsilon \\
C(c, j) &\triangleq \{0 \cdot x_0 + 0 \cdot x_0 - 1 \cdot x_j + 0 \cdot (x_0 \cdot x_0) + c = 0 \\
C(x^{\mathsf{F}}, j) &\triangleq \epsilon \\
C(x^{\mathsf{Bool}}, j) &\triangleq \{-1 \cdot x_j + 0 \cdot x_j + 0 \cdot x_0 + 1 \cdot (x_j \cdot x_j) + 0 = 0 \\
C(\hat{e}_1 / \hat{e}_2, w, j) &\triangleq \begin{cases} C(\hat{e}_1); C(\hat{e}_2); 0 \cdot x_{\iota(\hat{e}_2)} + 0 \cdot x_j - 1 \cdot x_{\iota(\hat{e}_1)} + 1 \cdot (x_{\iota(\hat{e}_2)} \cdot x_j) + 0 = 0 \\ 0 \cdot x_{\iota(\hat{e}_2)} + 0 \cdot x_w + 0 \cdot x_0 + 1 \cdot (x_{\iota(\hat{e}_2)} \cdot x_w) - 1 = 0 \end{cases} \\
C(\hat{e}_1 \dot{\vee} \hat{e}_2, j) &\triangleq \begin{cases} C(\hat{e}_1); C(\hat{e}_2); 1 \cdot x_{\iota(\hat{e}_1)} + 1 \cdot x_{\iota(\hat{e}_2)} - 1 \cdot x_j - 2 \cdot (x_{\iota(\hat{e}_1)} \cdot x_{\iota(\hat{e}_2)}) + 0 = 0 \\ -1 \cdot x_{\iota(\hat{e}_1)} + 0 \cdot x_{\iota(\hat{e}_1)} + 0 \cdot x_0 + 1 \cdot (x_{\iota(\hat{e}_1)} \cdot x_{\iota(\hat{e}_1)}) + 0 = 0 \\ -1 \cdot x_{\iota(\hat{e}_2)} + 0 \cdot x_{\iota(\hat{e}_2)} + 0 \cdot x_0 + 1 \cdot (x_{\iota(\hat{e}_2)} \cdot x_{\iota(\hat{e}_2)}) + 0 = 0 \end{cases} \\
C(\mathsf{isZero}\ \hat{e}, w, j) &\triangleq \begin{cases} C(\hat{e}); 0 \cdot x_{\iota(\hat{e})} + 0 \cdot x_j + 0 \cdot x_0 - 1 \cdot (x_{\iota(\hat{e})} \cdot x_j) + 0 = 0 \\ 0 \cdot x_{\iota(\hat{e})} + 0 \cdot x_w - 1 \cdot x_j - 1 \cdot (x_{\iota(\hat{e})} \cdot x_w) + 1 = 0 \end{cases} \\
C(\hat{e} \neq 0, w) &\triangleq \{C(\hat{e}); 0 \cdot x_{\iota(\hat{e})} + 0 \cdot x_w + 0 \cdot x_0 + 1 \cdot (x_{\iota(\hat{e})} \cdot x_w) - 1 = 0 \\
C(\hat{e} \in \mathbb{B}) &\triangleq \{C(\hat{e}); -1 \cdot x_{\iota(\hat{e})} + 0 \cdot x_{\iota(\hat{e})} + 0 \cdot x_0 + 1 \cdot (x_{\iota(\hat{e})} \cdot x_{\iota(\hat{e})}) + 0 = 0 \\
C(\hat{e}_1 \doteq \hat{e}_2) &\triangleq \{C(\hat{e}_1); C(\hat{e}_2); 1 \cdot x_{\iota(\hat{e}_1)} - 1 \cdot x_{\iota(\hat{e}_2)} + 0 \cdot x_0 + 0 \cdot (x_{\iota(\hat{e}_1)} \cdot x_{\iota(\hat{e}_2)}) + 0 = 0
\end{aligned}
$$

Fig. 10. Compilation

3.4 Translations

A PLINK program (containing an expression e and assertions as) is compiled down to a PLONK constraint system in two stages: first, it is labeled to obtained a list of PLINK-IR programs $\hat{p}s$, and then all the PLINK-IR programs are compiled to obtain a system of equations in the PLONK format.

When labeling a program, the compiler must ensure that all appearances of a variable use the same wire, as this is the way to ensure they will share the same value across the arithmetic circuit. To achieve this, a *labeling environment* Λ is kept with the wires that have been assigned to the variables.

The labeling process is described in fig. 9 using the judgment $\Lambda \vdash p \rightsquigarrow \hat{p}s \dashv \Lambda'$ to express that, under a labeling environment Λ, the program p is labeled to produce a list of labeled programs $\hat{p}s$ together with an updated environment Λ'. We overload the notation to also label expressions e and assertions a. In what follows, we denote by $\langle \hat{p} \rangle$ a list of labeled programs containing a single program $\hat{p}$ and by ϵ the empty list. Additionally, we write a sequential labeling of a list of assertions as with judgments like $\Lambda \vdash as \rightsquigarrow^* \hat{a}s \dashv \Lambda'$; which is useful for labeling the assertions of a top-level program.

Note in rule L-CONS that vector expressions are labeled component-wise and the original vector structure is forgotten in the resulting list of labeled expressions. This process of "flattening" non-scalar expressions is a first approximation to the constraint system, where the only possible values are field elements, and is also the reason why vectors are not included in the PLINK-IR syntax (fig. 5b). In contrast, note that every expression of *scalar* type gets labeled to produce a list of exactly one labeled expression; this invariant is needed to correctly implement the labeling function and guarantees that labeling does not "flatten" programs excessively.

Most labeling rules are straightforward: they recursively label subexpressions and use fresh wire indices when needed. For variables not present in the environment (rule L-VAR2), the freshly created index is added to the labeling environment. Aside from this, it is worth noting that rules L-TRUE, L-FALSE and L-EQ are responsible for removing the syntactic sugar from the PLINK language.

The labeling rules shown in fig. 9 can be proven to be deterministic, which motivates defining a labeling function ℓ that takes Plink programs and returns Plink-IR programs; concretely, $\ell(p)$ is defined to be the unique list of labeled programs $\hat{p}s$ that satisfies $\varnothing \vdash p \rightsquigarrow \hat{p}s \dashv \Lambda$ for some Λ.

The second stage of compilation consists of translating the list of labeled expressions to a constraint system. A representative subset of the translation rules is presented in fig. 10. In particular, the rules for addition, subtraction and multiplication are omitted as they are straightforward, and the ones for negation, disjunction and conjunction are similar to that for exclusive disjunction $\dot{\vee}$.

Note that wires and field-typed variables are only relevant when they are referenced in some other expression, but they do not introduce any constraints by themselves. The rule for boolean variables highlights the need of making Plink (and Plink-IR) a strongly typed language: since Plonk variables take values in $\mathbb{F}_p$, booleans must be encoded as 0 or 1. The Plink compiler achieves this by automatically adding the constraint $x_j \cdot (x_j - 1) = 0$; thus, the developer does not need to remember to add it manually. The rule for division shows why some labeled expressions need to have an additional *witness* wire (the rules for isZero and nonZero follow a similar pattern). Consider an expression like $\hat{e}_1/\hat{e}_2$ which is assigned the output wire j. Intuitively, there should be a constraint expressing $x_{\iota(\hat{e}_1)}/x_{\iota(\hat{e}_2)} = x_j$; since Plonk's constraints do not support division natively, we could rewrite it as $x_{\iota(\hat{e}_1)} = x_{\iota(\hat{e}_2)} \cdot x_j$, but this poses a problem when $x_{\iota(\hat{e}_1)} = x_{\iota(\hat{e}_2)} = 0$: the constraint is satisfied but the program has a division of zero by zero, so its semantics cannot be satisfied. The solution is to add an additional constraint enforcing that $x_{\iota(\hat{e}_2)}$ is non-zero, which can be achieved by introducing a "witness" wire w and asserting $x_{\iota(\hat{e}_2)} \cdot x_w = 1$.

3.5 Witness generation

While the semantics and the translations between different languages presented in §§ 3.3, 3.4 are all that is needed to generate a constraint system from a high-level Plink program, for practical use knowing the values that each wire takes (the *witnesses*) is helpful in understanding or debugging circuits. Furthermore, the prover's secret typically consists only of a small number of values and the values in most wires can be derived automatically from this secret witness, taking the burden of computing all witnesses off the prover.

In particular, the witnesses can be computed automatically from the values of the variables (given in the form of a variable valuation ρ). This computation—known as *witness generation*—effectively gives a second kind of semantics of the labeled intermediate representation than the "valuation-coherence" given in fig. 8. The main difference is that witness generation *returns* a wire valuation σ while valuation-coherence *determines* if a wire valuation σ is coherent with the program.

Witness generation may need user-provided *hints* for newly defined variables, which are expressed as Haskell functions that describe how to compute their values (see fig. 2). These hints are irrelevant for the creation of constraint systems and, as such, they are not checked to be correct; if they are not, the witness generator will simply fail to produce a witness satisfying all the constraints.

The witness generator witnessGen (fig. 11) takes a list of labeled programs $\hat{p}s$ and a valuation ρ and creates a wire valuation σ coherent with $\hat{p}s$, if possible. It does so by iterating an auxiliary function witnessGen′ that takes a single labeled program $\hat{p}$ and a valuation σ and extends σ with the information from $\hat{p}$. There are some cases where there is no coherent valuation; for instance, in the rule for the "≠0" assertion, when the value of the expression *is* zero. In these cases, the witness generator does not generate any witness and simply fails (with $\perp$).

4 Security Guarantees (Theorems)

In this section, we present the security properties that are guaranteed by the Plink language. In § 4.1 and § 4.2 we present the security results about the Plink compiler that have been mechanized in Liquid Haskell; together, they guarantee that the compiler preserves the semantics between the

$$\boxed{\mathsf{witnessGen} :: (\rho, \hat{p}^*) \to \sigma; \quad \mathsf{witnessGen}^*_\rho :: (\sigma, \hat{p}^*) \to \sigma; \quad \mathsf{witnessGen}'_\rho :: (\sigma, \hat{p}) \to \sigma}$$

$$\begin{array}{lcl}
\mathsf{witnessGen}(\rho, \hat{ps}) \triangleq \mathsf{witnessGen}^*_\rho\ \varnothing\ \hat{ps}\ \textit{where} & & \\
\quad \mathsf{witnessGen}^*_\rho\ \sigma\ \epsilon & \triangleq & \sigma \\
\quad \mathsf{witnessGen}^*_\rho\ \sigma\ (\hat{p} :: \hat{ps}) & \triangleq & \mathsf{witnessGen}^*_\rho\ (\mathsf{witnessGen}'_\rho\ \sigma\ \hat{p})\ \hat{ps} \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\mathsf{wire}_j) & \triangleq & \sigma \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (c, j) & \triangleq & \sigma, j \mapsto c \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (x^\tau, j) & \triangleq & \sigma, j \mapsto \rho(x^\tau) \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e}_1 \odot \hat{e}_2, j) & \triangleq & \sigma_2, j \mapsto \overline{\sigma_1}(\hat{e}_1) \odot_{\mathbb{F}_p} \overline{\sigma_2}(\hat{e}_2) \\
\qquad \textit{where} \odot \in \{+, -, *\}\ ;\ \sigma_1 \triangleq \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e}_1\ ;\ \sigma_2 \triangleq \mathsf{witnessGen}'_\rho\ \sigma_1\ \hat{e}_2 & & \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e}_1 \odot \hat{e}_2, j) & \triangleq & \sigma_2, j \mapsto \overline{\sigma_1}(\hat{e}_1)\big|_{\{0,1\}} \odot_{\mathbb{F}_p} \overline{\sigma_2}(\hat{e}_2)\big|_{\{0,1\}} \\
\qquad \textit{where} \odot \in \{\wedge, \vee, \dot{\vee}\}\ ;\ \sigma_1 \triangleq \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e}_1\ ;\ \sigma_2 \triangleq \mathsf{witnessGen}'_\rho\ \sigma_1\ \hat{e}_2 & & \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e}_1 / \hat{e}_2, w, j) & \triangleq & \sigma_2, j \mapsto \overline{\sigma_1}(\hat{e}_1) \mathbin{//} \overline{\sigma_2}(\hat{e}_2), w \mapsto 1 \mathbin{//} \overline{\sigma_2}(\hat{e}_2) \\
\qquad \textit{where}\ \sigma_1 \triangleq \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e}_1\ ;\ \sigma_2 \triangleq \mathsf{witnessGen}'_\rho\ \sigma_1\ \hat{e}_2 & & \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\mathsf{isZero}\ \hat{e}, w, j) & \triangleq & \sigma', j \mapsto \mathsf{if}\ \overline{\sigma'}(\hat{e}) = 0\ \mathsf{then}\ 1\ \mathsf{else}\ 0, w \mapsto 1 \mathbin{//} \overline{\sigma'}(\hat{e}) \\
\qquad \textit{where}\ \sigma' \triangleq \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e} & & \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e} \neq 0, w) & \triangleq & \sigma', w \mapsto 1 \mathbin{//} \overline{\sigma'}(\hat{e})\ \textit{where}\ \sigma' \triangleq \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e} \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e} \in \mathbb{B}) & \triangleq & \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e} \\
\quad \mathsf{witnessGen}'_\rho\ \sigma\ (\hat{e}_1 \doteq \hat{e}_2) & \triangleq & \mathsf{witnessGen}'_\rho\ (\mathsf{witnessGen}'_\rho\ \sigma\ \hat{e}_1)\ \hat{e}_2 \\
\quad x \mathbin{//} y & \triangleq & \mathsf{if}\ y \neq 0\ \mathsf{then}\ x /_{\mathbb{F}_p} y\ \mathsf{else}\ \bot \\
\quad x\big|_{\{0,1\}} & \triangleq & \mathsf{if}\ x \in \{0,1\}\ \mathsf{then}\ x\ \mathsf{else}\ \bot
\end{array}$$

Fig. 11. Witness Generation for the PLINK-IR Language ($\bot$ means the witness generator globally aborts).

high-level program and the constraint system. In § 4.3, we show how developers can leverage Liquid Haskell's refinement type system to formally specify and verify properties of PLINK programs.

4.1 Semantics Preservation

Intuitively, theorem 1 states that compilation from PLINK-IR to PLONK constraint systems is correct in the sense that every labeled program introduces the same constraints as its compilation.

Theorem 1. For each labeled program $\hat{p}$ and wire valuation σ, we have $\sigma \vdash \hat{p} \iff \sigma \vdash C(\hat{p})$.

4.2 Correctness of Labeling

Theorem 2 formalizes the idea that labeling (the translation from PLINK to PLINK-IR) is done correctly; intuitively, this means that the values of high-level PLINK expressions are propagated to the corresponding output wires. The theorem needs a previous lemma:

Lemma 1. Let e be a scalar expression. Assume that (1) $\Lambda \vdash e \rightsquigarrow \langle \hat{e} \rangle \dashv \Lambda'$, (2) $\sigma' = \mathsf{witnessGen}'_\rho\ \sigma\ \hat{e}$, and (3) $\rho\big|_{\mathrm{dom}(\Lambda)} = \sigma \circ \Lambda\big|_{\mathrm{dom}(\Lambda)}$ (intuitively, valuations ρ and σ agree on the variables labeled in Λ). Then, we have (1) $\rho \vdash e \Downarrow v \iff \overline{\sigma}(\hat{e}) = v$ and (2) $\rho\big|_{\mathrm{dom}(\Lambda')} = \sigma' \circ \Lambda'\big|_{\mathrm{dom}(\Lambda')}$.

Theorem 2 (Correctness of Labeling)**.** Let e be a scalar expression and ρ a variable valuation. Then, if $\langle \hat{e} \rangle = \ell(e)$ and $\sigma = \mathsf{witnessGen}(\rho, \langle \hat{e} \rangle)$, we have $\rho \vdash e \Downarrow v \iff \overline{\sigma}(\hat{e}) = v$. Consequently, for any assertion a, if $\hat{as} = \ell(a)$, then $\rho \vdash a \iff \sigma \vdash \hat{as}$.

4.3 Formal Specification & Verification

Plink is implemented as an embedded DSL in (Liquid) Haskell; as such, programs written in Plink are just Haskell values and therefore can be verified using Liquid Haskell. The exact properties that are specified and proven about Plink programs are application-dependent and can be defined by the user as needed, making this approach applicable for a broad variety of verification properties. We present below five types of properties that we proved in our development.

(1) **Types of expressions**: Plink's compilation function has the precondition that the expressions it takes are well-typed. Therefore, the user is required to prove well-typedness of their programs by writing refinement type annotations that are then checked by Liquid Haskell. For instance, the following type annotation says that the "equality checking" operator takes two field elements and returns a single boolean element: `{-@ (=?) :: FieldDSL p -> FieldDSL p -> BoolDSL p @-}`.

(2) **Statements about values of expressions**: for instance, we prove, using deep verification, that under any valuation ρ, any expression of type `TBool` can only take values in $\{0, 1\}$:

```
{-@ booleanity :: ρ:NameValuation p -> e:BoolDSL p
               -> {v:DSLValue p | eval e ρ = Just v} -> {v = VF 0 || v = VF 1} @-}
```

Indeed, theorem 1 and theorem 2 propagates this property to the constraints: the variable corresponding to a boolean expression in the constraint system can only take the values in $\{0, 1\}$.

(3) **Full functional correctness**, by writing arbitrary theorems about functions and prove them externally. For instance, we proved that the result of "`rotateR`" is a permutation of its input and how exactly the elements are swapped around:

```
{-@ rotateSpec :: τ:Ty -> xs:VecDSL p τ -> n:Btwn 0 (vlen xs) -> i:Btwn 0 (vlen xs)
               -> { get τ (rotateR τ xs n) i =
                    get τ xs ((i + vlen xs - n) mod (vlen xs)) } @-}
```

This property also propagates to the constraint system: for example, if `vec` is a vector of length 3, `get TF (rotateR TF vec 1) 2` and `get TF vec 1` will have the same wire in the constraint system.

(4) **Domain-specific pre- and post-conditions**: for instance, the main compression loop of the SHA-256 algorithm processes its input in *blocks* of length 512; thus, the algorithm defines a *preprocessing* phase that pads the input with additional bits to ensure the resulting length is a multiple of 512. Part of this padding is the length of the input message encoded as a 64-bit integer, so the input must be a bitstring of less than 2^{64} bits. Both of these properties (the *precondition* that the message is not too long and the *postcondition* stating that the resulting length is a multiple of 512) can be written as a specification of the function to be automatically checked at compile time:

```
{-@ padding :: msg:{VecDSL p TBool | vlen msg < pow 2 64}
            -> {res:VecDSL p TBool | (vlen res) mod 512 = 0} @-}
```

(5) **Consistency of internal data**: for instance, in the reference implementation of the Poseidon2 hash function [HorizenLabs 2024], the parameter `d` admits only values in $\{3, 5, 7\}$; however, the data validator additionally admits the value `d` = 11 by mistake, allowing for an invalid internal state. With Liquid Haskell, on the other hand, we defined the type for the valid values of `d`, `{-@ type ValidD = {d:Int | d = 3 || d = 5 || d = 7} @-}`, which enforces consistency.

5 Implementation

In this section, we present the implementation details of the Plink language and motivate the main differences between the implemented compiler and the theoretical model described in this paper.

The Plink language has been implemented as an *embedded* [Prinz et al. 2022] Domain Specific Language (eDSL) in Liquid Haskell (i.e. Plink programs are just Haskell expressions) and the compiler is implemented in ~1500 lines of code plus Liquid Haskell annotations. These annotations

```
{-@ rotateR :: τ:Ty -> u:VecDSL p τ -> Btwn 0 (vlen u)
            -> {v:VecDSL p τ | vlen v = vlen u} @-}
rotateR :: Ty -> DSL p -> Int -> DSL p
rotateR τ xs n = let (ys, zs) = vTakeDrop τ (vlen xs - n) xs in vAppend τ zs ys

{-@ vTakeDrop :: τ:Ty -> n:Nat -> u:{VecDSL p τ | vlen u >= n} ->
       ({v:VecDSL p τ | vlen v = n}, {w:VecDSL p τ | vlen w = (vlen u) - n}) @-}
```

Fig. 12. PLINK implementation of `rotateR`

perform *light* verification [Vazou et al. 2018] of the PLINK compiler. For example, the labeling process keeps track of the number m of labels that have been assigned so far, and the resulting PLINK-IR program is proven to use as wire indices only integers between 0 and $m-1$; this way, the wire valuation $\sigma : \mathbb{N} \to \mathbb{F}_p$ can be interpreted as a list of length m, and indexing is proven to be bound-safe.

Aside from these simple verification properties, the lemmata and theorems stated in this paper have been mechanized using *deep* verification, that is, they have been stated and proven as independent functions. All these proofs together take up ~880 lines of Haskell code with Liquid Haskell annotations and verification takes ~18 minutes.

These deep proofs do not rely on the theory of finite fields, which is not currently supported by Liquid Haskell; thus, PLINK finite fields are not represented as logical finite field elements (even though such a representation [Ozdemir et al. 2023] would be desirable in the future). Yet, we claim that this representation is not relevant for the correctness of the compiler, that merely requires that the semantics (fig. 7) and valuation coherence (fig. 8) behave equivalently to the satisfiability of compilation (fig. 10), without the need to interpret the arithmetic operations; such an interpretation would only be required to prove soundness of PLONK itself, which is out of the scope of this work.

5.1 Standard library

PLINK comes with a standard library written in Haskell and verified using Liquid Haskell that implements functions commonly used in the development of cryptographic programs, such as addition modulo powers of 2, component-wise operations on vectors and logical shifts and rotations on bitstrings. This library takes ~300 lines of Liquid Haskell code and verifies in ~15 seconds.

As an example, fig. 12 shows the library implementation of the `rotateR` function (that shifts the input vector to the right, wrapping around so that the removed elements are appended at the left) in PLINK with the specification of its auxiliary function (`vAppend` was already defined in fig. 3).

Notice how the *logic* of these functions matches exactly that of a Haskell implementation—recursion, pattern matching, etc.—, so that the only difference is the Liquid Haskell annotations and the additional "type" parameter. Liquid Haskell annotations are used to formally specify properties of the implementation to be checked statically. For instance, `rotateR` **requires** that its integer argument is a natural number less than the length of the vector and **ensures** that its output is the same length as its input. The type parameter τ is used to implement parametric polymorphism in PLINK: because the type system is implemented as Liquid Haskell refinements, and this requires that values that appear in specifications are explicitly defined beforehand, PLINK polymorphism is explicit.

As a final remark, the top-level programs p described in fig. 5a are implemented as a monad called `PlinkST` (cf. fig. 2) that wraps any value (most often, a PLINK expression e) and carries additional information about the program; this consists mainly of the list of assertions a^* that have been placed so far, but it also includes the hints to the witness generator as described in § 2.3.

5.2 Optimizations

In the Plonk zkSNARK, *proving time* and *circuit preprocessing time* grow with the number of arithmetic gates in the circuit [Gabizon et al. 2019]; for this reason, there is a practical interest in minimizing the size of circuits, either by an optimization pass in the compiler [Stronati et al. 2024] or by simplifying the final constraint systems themselves [Albert et al. 2022; Ambrona et al. 2022].

The Plink compiler implements two optimization strategies to transform high-level programs to more efficient ones (introducing fewer constraints): *inlining* and *common subexpression elimination*.

5.2.1 Inlining. Plonk-style constraints allow to encode simultaneously addition, multiplication of expressions by constants and multiplication of two expressions. However, the direct approach to compilation (per § 3.4) does not take full advantage of this expressive power, producing more equations than needed. For example, the expression $2 * x + y$ would produce a system of three constraints, when it can be encoded optimally with a single constraint as $2 \cdot x_1 + 1 \cdot x_3 - 1 \cdot x_4 + 0 \cdot (x_1 \cdot x_3) + 0 = 0$ by treating it as a linear combination.

Automatically performing an optimization like this requires special treatment of certain expressions, like linear combinations. The Plink compiler indeed makes a pass coalescing specific combinations of basic arithmetic operations into more specialized ones, which are implemented with ad-hoc terms in the syntax. This inlining pass is in the first phase of the compiler; it is preferable that it happens early on in the pipeline—before labeling—so the compiler handles simpler expressions and avoids creating unnecessary wire labels. Additionally, the compiler runs *constant folding* in parallel, since both optimizations can benefit from each other.

Further, we prove the following security result about correctness of optimizations:

Theorem 3 (Optimization Correctness)**.** Let e be a Plink expression and ρ a valuation. Then, if there is a v such that $\rho \vdash e \Downarrow v$, then also $\rho \vdash opt(e) \Downarrow v$, where opt is the inlining optimization.

5.2.2 Common Subexpression Elimination (CSE). CSE [Cocke 1970] is a standard optimization pass in compilers that reduces unnecessary re-computation of available data. In arithmetic circuits, it can be implemented by sharing a single output wire among equal subexpressions, so that the constraints are added once and used multiple times. This sharing is possible because all subexpressions are assigned an output wire; for this reason, CSE cannot be applied before labeling, which makes its implementation fundamentally different from inlining.

In the Plink implementation, CSE is applied during the labeling phase, by extending the labeling environment Λ to include all expressions, instead of only variables. Because CSE significantly complicates the labeling process as well as the proof of theorem 2, it has neither been included in the theoretical model nor is it part of the formalization of the correctness labeling theorem.

6 Benchmarks

To evaluate the expressiveness of the Plink language, we use it to implement and partially verify two popular hash functions in the area of ZK proof systems, SHA-256 and Poseidon2. These benchmarks are especially relevant because hash functions are used very frequently in zkSNARKs, as they are an important part of many popular applications, such as rollups (where they are used for verifying Merkle trees) or signatures validity (where they are used to hash messages); further, these two hash functions were defined with different goals and following different strategies, so they offer good opportunities to benchmark the Plink compiler: Poseidon2 was designed with the explicit goal of being *arithmetization-friendly* (based solely on arithmetic operations over the native finite field), while SHA-256 was designed to be efficiently computable on 32-bit processors, so it poses unique problems for finite field arithmetization.

Regarding formal verification, hash functions present a challenge: by design, there is no simple relation between their inputs and outputs, so formally specifying their behavior means re-implementing the function in the specification language. Still, it is possible to formally verify their basic building blocks, which are combined in a functional way helping reduce the change of errors.

Concretely, in our implementation both hash functions are written using functions from the standard library. As mentioned in § 5.1, the standard library contains formal specifications that are enforced by the compiler. This alone increases the security guarantees of the programs implemented using it, but it also helps in formally verifying some properties of these programs by providing useful postconditions that can be leveraged in the proofs. As an additional safeguard, the witness generator can help in debugging an incorrect implementation: since the whole list of witnesses (including the "output" values of the circuit) is available to the developer, it can be checked against the expected result of the hash function, which can be obtained from a standard implementation.

6.1 SHA-256

Our implementation of SHA-256 in PLINK consists of around 250 lines of Liquid Haskell code (160 for the logic and 90 for the definition of numeric constants) and makes use of many of the library functions described in § 5 since the SHA-256 algorithm is heavily based on 32-bit arithmetic. By using these functions as building blocks, it can be written in a completely functional way: in fact, the PLINK implementation mirrors closely the implementation one would write in Haskell.

Even if, as discussed above, the input-output specification of SHA-256 is impractical, there are some interesting pre- and post-conditions for some of the components that can be stated and proven in PLINK. To give a concrete example, SHA-256 works by splitting its input message into chunks of 512 bits, or 16 32-bit words; then, each chunk is extended to a list of 64 words which is compressed by using each of the words to sequentially update a list of 8 variables. It can be challenging to keep track of these various lengths and to ensure that each of the components returns a list of the size that the next one expects. Refinement types offer a powerful framework to address this kind of situations, by formally specifying the lengths of the lists that each function receives and returns and having the compiler verify that the implementation indeed satisfies these requirements.

For example, the refinement type signatures for `extend` and `vChunk` would be the following:

```
{-@ extend :: ListN (Word p) 16 -> PlinkST p (ListN (Word p) 64) @-}
{-@ vChunk :: τ:Ty -> n:Nat1 -> v:{VecDSL p τ | (vlen v) mod n = 0}
           -> {l:[{w:VecDSL p τ | vlen w = n}] | n * len l = vlen v} @-}
```

These refinements express—along with the PLINK type signatures—the length requirements of these functions. Verification of the whole SHA-256 implementation takes around 20 seconds.

Finally, we evaluate the effectiveness of the PLINK compiler optimizations—inlining and CSE—on our SHA-256 implementation. The length of the input message has an impact on the size of the final constraint system: the more 512-bit blocks there are in the message, the more constraints there are in the system, which translates to a longer running time for the prover. We present in fig. 13(a) the number of constraints for inputs of 1–7 blocks and all four combinations of the two implementation strategies we implement. We can draw three conclusions from this graph:

(1) The inlining optimization is very efficient in reducing the number of constraints: just on its own, it reduces the number of constraints by around 45%.

(2) CSE is efficient on its own (around 25% reduction) but it has only a small effect when applied in combination with inlining (it shows a 3.1% improvement with respect to applying only inlining).

(3) The number of constraints grows linearly with the number of blocks across all combinations. While unsurprising for the unoptimized programs, this shows that there is only a constant chance of creating redundant constraints that can be amortized by sharing wires through CSE.

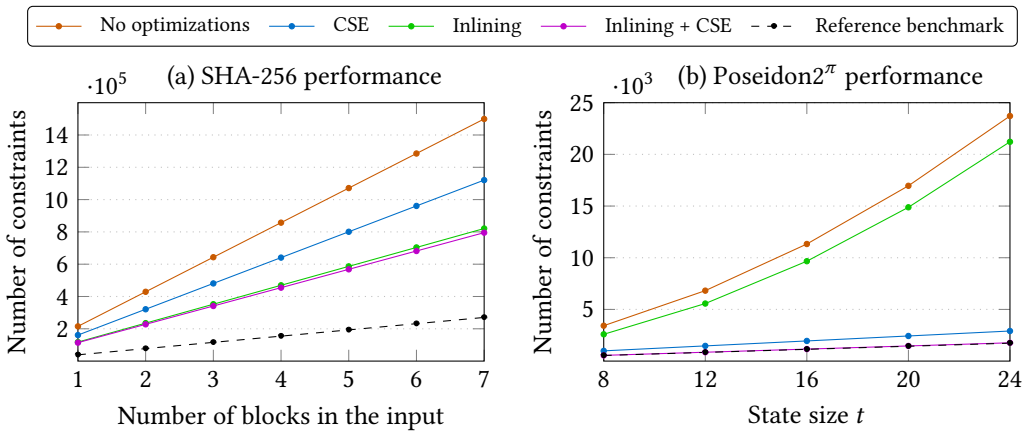


Fig. 13. Size of the constraint system generated by Plink for our benchmarks (lower is better)

Finding external benchmarks for comparison can be challenging because there is little published material on the topic and many industrial applications are using extended implementations of Plonk that support different styles of constraints. One benchmark using the same constraints as us [Aztec 2024] produced a system with 38799 constraints for inputs of one block. This result (extrapolated linearly to higher numbers of blocks) is shown for comparison in fig. 13(a); the Plink compiler with full optimizations produces around three times as many constraints for SHA-256, which we consider an acceptable factor as efficiency is not the main objective of this work.

6.2 Poseidon2

Poseidon2 [Grassi et al. 2023] is a closely related family of hash functions indexed, among other parameters, by the size t of its output, and based on a permutation over $\mathbb{F}_p^t$ called Poseidon2$^\pi$ (i.e. a bijection $\mathbb{F}_p^t \to \mathbb{F}_p^t$). In this benchmark we focus on analyzing Poseidon2$^\pi$, as is commonly done, since using it to write a complete hash function is mostly language-independent. Our implementation of the permutation takes up about 170 lines of Liquid Haskell code, plus an additional 970 lines for defining the numeric constants necessary for a collection of particular instances. Overall, the verification time is around 60 seconds.

The Poseidon2$^\pi$ permutation consists of a succession of *rounds* that modify the *state* $\in \mathbb{F}_p^t$, where in each round (1) a number of constants is added to the state, (2) some or all of the coordinates of the state are raised to a fixed exponent d, and (3) the state is multiplied by a $t \times t$ matrix (each round uses the *external* matrix $M_{\mathcal{E}}$ or the *internal* one $M_{\mathcal{I}}$). Unlike SHA-256, Poseidon2$^\pi$ was designed for use in zkSNARKs, so it allows for a high degree of optimization towards reducing the amount of constraints. For example, in step (2) above, the authors recommend using a square-and-multiply scheme to implement exponentiation; Plink's optimizations can achieve the same number of constraints with little user effort. Moreover, the authors give concrete suggestions [Grassi et al. 2023, App. B] on how to compute the product of the state vector with the matrix in step (3). For instance, they show how their strategy allows for encoding multiplication by their chosen 4×4 external matrix in just 8 Plonk constraints. Their technique can be implemented directly in Plink, obtaining the same performance after optimizations, as shown in table 1. This table also shows the number of constraints for different values of t and all combinations of optimizations.

Finally, fig. 13(b) shows how the number of Plonk constraints increases relative to the state size t in comparison with the optimal value from [Grassi et al. 2023, Fig. 3]. We note that these numbers

Table 1. Number of constraints for the computation of $M_{\mathcal{E}} \cdot x$

State size	$t = 4$	$t = 8$	$t = 12$	$t = 16$	$t = 20$
No optimizations	22	164	324	536	800
CSE	8	33	49	65	81
Inlining	22	148	300	504	760
Inlining + CSE	8	28	44	60	76

include only the constraints relative to matrix multiplications for an entire run of the permutation (all internal and external rounds, combined), as the constraints introduced in the exponentiation step are highly dependent on the parameter d. From this data, we can conclude:

(1) CSE achieves close to optimal performance when applied to a circuit that has been designed for constraint efficiency, since it aligns with the intention of circuit designers of reusing as many computations as possible. In fact, CSE reduces the growth in constraints from quadratic to only linear.

(2) Inlining does not help much by itself, but it does close the gap between the numbers attained through CSE and the claimed optimal value. We note that this behavior is reversed with respect to that obtained in SHA-256, where inlining was the more effective optimization.

7 Related work

In this section, we review the previous work that is most closely related to our approach.

Formal verification in cryptography. Formal methods have been applied in areas of cryptography for formalizing security proofs in proof assistants. As notable examples, the EasyCrypt interactive framework [Barthe et al. 2014] formally specifies and verifies the security of cryptographic protocols, by modeling them as probabilistic programs; Fiat Crypto [Erbsen et al. 2019] provides a technique for performing cryptographic arithmetic in high-level code with Rocq machine-checked proofs; and ProvenZK [Reilabs 2025] is a library of the Lean theorem prover that mechanizes the cryptographic security of ZK protocols. Such works aim to mechanize the security of cryptographic protocols, while PLINK assumes the soundness of PLONK and propagates it closer to the user.

Formal methods in Zero-Knowledge. Formal methods have been used in several ways within the scope of ZK. For example, Laufer et al. [2024] show how to construct a ZK proof of knowledge of a Lean proof, Albert et al. [2022] use formal reasoning to reduce the time and space cost of constructing ZK proofs by reducing the number of constraints, and Isabel et al. [2024]; Pailoor et al. [2023] mechanically verify the correctness of the constraint systems. Such techniques are complementary to PLINK and can be adapted to improve our approach.

DSLs for ZK systems. Due to the increasing popularity of ZK proofs and to the expanding scope of their applications, numerous DSLs have been developed in the past few years to help in their construction. ZoKrates [Eberhardt and Tai 2018] is a mature DSL, developed in Rust, that compiles to a Rank-1 Constraint System (R1CS) originally built to generate Solidity smart contracts. Circom [Bellés-Muñoz et al. 2022] is similarly developed in Rust and emits R1CS, but it additionally generates witness generator functions in both C++ and WebAssembly. Noir [Aztec 2025] is a recent, rapidly evolving DSL, which is also developed in Rust, but compiles to a higher level circuit representation, called Abstract Circuit Intermediate Representation (ACIR), aimed to target multiple backends. CirC [Ozdemir et al. 2022] is a DSL implemented in (originally Haskell and then) Rust that compiles to R1CS, but also SMT (used for verification and bug-finding), and ILP (used for optimization). Lurk [Amin et al. 2023] is a Turing complete, LISP-like programming

Table 2. Comparison of Plink with other verified DSLs for ZK. Verification can be that the *Compiler* is correct, the Constraint System (CS) adheres to a specification, or *Functional* correctness of the high-level program.

DSL	Target CS	Written in	Verification	Verified using
Coda	R1CS	OCaml (eDSL)	Functional	Rocq
Leo	R1CS	Rust	Compiler + CS	ACL2
Clap	Plonk	Rust (eDSL)	Compiler	Agda
Plink	Plonk	**Liquid Haskell (eDSL)**	**Compiler + Functional**	**Liquid Haskell**

language that compiles to down to R1CS. Though carefully designed, none of these DSLs provides a formal correctness guarantee for the generated constraint systems.

Closely related to Plink, there are three formally verified compilers that stand out. Namely, Coda [Liu et al. 2024], Leo [Provable Inc 2025], and Clap [Stronati et al. 2024], whose comparison with Plink is summarized in table 2. In the verification column, we distinguish between (1) **functional** correctness when verification is performed on the high-level programs that describe the intended arithmetic circuit, (2) **compiler** correctness when the translation from high-level programs down to the algebraic constraint system is verified à la CompCert [Leroy 2009], and (3) **constraint system (CS)** correctness when the generated CS is certified to adhere to a given specification.

Coda [Liu et al. 2024] is a high-level, functional DSL with native support for refinement types. Coda's type system is highly expressive (permitting the specification of number-theoretic properties of finite field elements), which means that verification cannot be done fully automatically; instead, it generates Rocq proof obligations to be discharged by the user. Even though both Coda and Plink rely on refinement types, they differ in two main ways. First, as the Plink language is embedded in Liquid Haskell, refinement types can be used to prove correctness of the *compiler* itself as well as about the high-level programs written in Plink. Second, instead of developing a new refinement type system, we rely on Liquid Haskell's, which is well-established and decidable. As a result, the verification of Plink programs is done automatically by the Liquid Haskell's SMT solver, without the need for user provided Rocq proofs.

A different approach is that of Leo or Clap: both DSLs focus on certifying that the compiler is correct, which is proven by modeling the implementation in a proof assistant and proving correctness of the model. In comparison, Plink relies on Liquid Haskell being both an implementation language and a theorem prover, so there is no need to model the implementation externally and thus no gap between the actual compiler and the object that is proven secure, minimizing the chance of errors.

8 Conclusions

We presented Plink, a new formally verified, strongly-typed, and declarative DSL for describing relations to be proved with the Plonk zkSNARK. Plink programs are written in Liquid Haskell and the developers can use its refinement types to verify functional correctness properties. The compiler then translates the high-level Plink programs into Plonk's algebraic constraint systems. We mechanically proved that the soundess of the Plonk protocol is propagated to the Plink programs by this compilation and also that the functional correctness properties are satisfied by the generated constraints. Additionally, the Plink compiler implements two optimization strategies for producing smaller constraint systems, one of which is also mechanically proven to be correct. We implemented and partially verified in Plink the Poseidon2 and SHA-256 hash functions, showing that Plink is suitable for encoding commonly used cryptographic functions.

References

Elvira Albert, Marta Bellés-Muñoz, Miguel Isabel, Clara Rodríguez-Núñez, and Albert Rubio. 2022. Distilling Constraints in Zero-Knowledge Protocols. In *Computer Aided Verification (CAV)*.

Aleo Network. 2025. Aleo Developer Documentation. https://developer.aleo.org/ Accessed 2025/07/05.

Miguel Ambrona, Anne-Laure Schmitt, Raphael R. Toledo, and Danny Willems. 2022. New optimization techniques for PlonK's arithmetization. Cryptology ePrint Archive, Paper 2022/462. https://eprint.iacr.org/2022/462

Nada Amin, John Burnham, François Garillot, Rosario Gennaro, Chhi'mèd Künzang, Daniel Rogozin, and Cameron Wong. 2023. LURK: Lambda, the Ultimate Recursive Knowledge (Experience Report). In *International Conference on Functional Programming (ICFP)*. doi:10.1145/3607839

Aztec. 2024. zk_bench: Benchmark circom and noir on some standard primitives from circomlib/noir stdlib. https://github.com/noir-lang/zk_bench Accessed 2025/07/08.

Aztec. 2025. Noir Documentation. https://noir-lang.org/ Accessed 2025/07/08.

David Balbás, Dario Fiore, Maria Isabel González Vasco, Damien Robissout, and Claudio Soriente. 2023. Modular Sumcheck Proofs with Applications to Machine Learning and Image Processing. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. doi:10.1145/3576915.3623160

barryWhiteHat. 2018. roll_up: Scale Ethereum with SNARKs. https://github.com/barryWhiteHat/roll_up Accessed 2025/07/07.

Gilles Barthe, François Dupressoir, Benjamin Grégoire, César Kunz, Benedikt Schmidt, and Pierre-Yves Strub. 2014. EasyCrypt: A Tutorial. In *Foundations of Security Analysis and Design VII (FOSAD)*. doi:10.1007/978-3-319-10082-1_6

Marta Bellés-Muñoz, Jordi Baylina, Vanesa Daza, and José L. Muñoz-Tapia. 2022. New Privacy Practices for Blockchain Software. *IEEE Software* (2022). doi:10.1109/MS.2021.3086718 https://ieeexplore.ieee.org/document/9446757.

Eli Ben Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. 2014. Zerocash: Decentralized Anonymous Payments from Bitcoin. In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP.2014.36

Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. 2020. Mina: Decentralized Cryptocurrenty at Scale. https://minaprotocol.com/wp-content/uploads/technicalWhitepaper.pdf Accessed 2025/07/05.

Stefanos Chaliasos, Jens Ernstberger, David Theodore, David Wong, Mohammad Jahanara, and Benjamin Livshits. 2024. SoK: What don't we know? Understanding Security Vulnerabilities in SNARKs. In *USENIX Conference on Security Symposium*. doi:10.48550/ARXIV.2402.15293

John Cocke. 1970. Global Common Subexpression Elimination. In *Symposium on Compiler Optimization*. doi:10.1145/390013.808480

Jacob Eberhardt and Stefan Tai. 2018. ZoKrates - Scalable Privacy-Preserving Off-Chain Computations. In *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*. doi:10.1109/Cybermatics_2018.2018.00199

Andres Erbsen, Jade Philipoom, Jason Gross, Robert Sloan, and Adam Chlipala. 2019. Simple High-Level Code for Cryptographic Arithmetic - With Proofs, Without Compromises. In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP.2019.00005

Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Paper 2019/953. https://eprint.iacr.org/2019/953 https://eprint.iacr.org/2019/953.

Christopher Goes, Awa Sun Yin, and Adrian Brink. 2022. Anoma: a unified architecture for full-stack decentralised applications. https://media.githubusercontent.com/media/anoma/whitepaper/refs/heads/main/whitepaper.pdf Accessed 2025/07/05.

Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. 2023. Poseidon2: A Faster Version of the Poseidon Hash Function. In *International Conference on Cryptology (AFRICACRYPT0)*. doi:10.1007/978-3-031-37679-5_8

HorizenLabs. 2024. Poseidon2 Hash Function implementation. https://github.com/HorizenLabs/poseidon2 Accessed 2025/11/05.

M. Isabel, C. Rodríguez-Núñez, and A. Rubio. 2024. Scalable Verification of Zero-Knowledge Protocols. In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP54263.2024.00133

Ranjit Jhala and Niki Vazou. 2021. Refinement Types: A Tutorial. In *Foundations and Trends in Programming Languages*. doi:10.1561/2500000032

Lagrange Labs. 2025. DeepProve: Framework to prove inference of ML models blazingly fast. https://www.lagrange.dev/deepprove Accessed 2025/07/08.

Evan Laufer, Alex Ozdemir, and Dan Boneh. 2024. zkPi: Proving Lean Theorems in Zero-Knowledge. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. doi:10.1145/3658644.3670322

Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing*. doi:10.1109/TDSC.2023.3348760

Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. In *Commun. ACM*. doi:10.1145/1538788.1538814

Junrui Liu, Ian Kretz, Hanzhi Liu, Bryan Tan, Jonathan Wang, Yi Sun, Luke Pearson, Anders Miltner, Isil Dillig, and Yu Feng. 2024. Certifying Zero-Knowledge Circuits with Refinement Types . In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP54263.2024.00078

Toghrul Maharramov, Dmitry Khovratovich, and Emanuele Francioni. 2021. The Dusk Network Whitepaper. https://dusk-cms.ams3.digitaloceanspaces.com/The_Dusk_Network_Whitepaper_v3_0_0_42ac204d48.pdf Accessed 2025/07/05.

Matter Labs. 2023. zkSync Documentation. https://docs.zksync.io/ Accessed 2025/07/05.

Midnight Network. 2024. Nightpaper: A litepaper introducing Midnight. https://midnight.network/whitepaper Accessed 2025/07/05.

Alex Ozdemir, Fraser Brown, and Riad S. Wahby. 2022. CirC: Compiler infrastructure for proof systems, software verification, and more. In *IEEE Symposium on Security and Privacy (S&P)*. doi:10.1109/SP46214.2022.9833782

Alex Ozdemir, Gereon Kremer, Cesare Tinelli, and Clark Barrett. 2023. Satisfiability Modulo Finite Fields. In *Computer Aided Verification (CAV)*. doi:10.1007/978-3-031-37703-7_8

Shankara Pailoor, Yanju Chen, Franklyn Wang, Clara Rodríguez, Jacob Van Gaffen, Jason Morton, Michael Chu, Brian Gu, Yu Feng, and Isil Dillig. 2023. Automated Detection of Underconstrained Circuits for Zero-Knowledge Proofs. In *Programming Languages Design and Implementation (PLDI)*. doi:10.1145/3591282

Polygon Labs. 2023. Polygon zkEVM Documentation. https://docs.polygon.technology/zkEVM/ Accessed 2025/07/05.

Jacob Prinz, G. A. Kavvos, and Leonidas Lampropoulos. 2022. Deeper Shallow Embeddings. In *International Conference on Interactive Theorem Proving (ITP)*. doi:10.4230/LIPIcs.ITP.2022.28

Provable Inc. 2025. Leo: Zero-Knowledge Programming Language. https://www.leo-lang.org/ Accessed 2025/07/05.

Reilabs. 2025. ProvenZK. https://github.com/reilabs/proven-zk Accessed 2025/07/05.

Masahiro Sakai. 2021. finite-field: Finite Fields. hackage. https://hackage.haskell.org/package/finite-field-0.10.0

Marco Stronati, Denis Firsov, Antonio Locascio, and Benjamin Livshits. 2024. Clap: a Semantic-Preserving Optimizing eDSL for Plonkish Proof Systems. arXiv:2405.12115 [cs.CR] https://arxiv.org/abs/2405.12115

Niki Vazou, Joachim Breitner, Rose Kunkel, David Van Horn, and Graham Hutton. 2018. Theorem proving for all: equational reasoning in Liquid Haskell (functional pearl). In *Haskell Symposium*. doi:10.1145/3299711.3242756

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement types for Haskell. In *International Conference on Functional Programming (ICFP)*. doi:10.1145/2692915.2628161